[12:31 pm, 23/10/2026] +91 80156 16233:
==========

STOCKSENSE - SMALL-RETAIL REORDER PLANNER | QUANTIX 2K26 | PS11

MASTER BUILD PROMPT

==========

ROLE

You are a senior full-stack engineer and applied data scientist. Build a production-quality, demo-ready web application called "StockSense" end to end. Work phase by phase (see PLAN). After each phase, summarize what was built, list files changed, and confirm the acceptance checks passed before moving on. Ask me nothing unless blocked; state assumptions and proceed.

---

1. PROBLEM

Small store owners (kirana, pharmacy, bakery, stationery, grocery) order stock from memory. Result:

- Fast-moving items stock out -> lost sales and lost customers.
- Slow or short-dated items pile up -> cash locked, expiry write-offs.

GOAL

The owner uploads past sales and enters, per product: stock on hand, supplier lead time, shelf life. The tool forecasts demand per product until the next delivery and produces a RANKED reorder list with EXACT unit quantities, so the most urgent and most profitable items come first. It must respect supplier lead time and shelf life.

PRIMARY USER: a non-technical shop owner on a phone or low-end laptop. Design for 2-minute daily use. No jargon in the UI ("Order 48 units - you will run out in 2 days" beats "ROP breached").

---

2. TECH STACK (fixed)

Frontend : React + TypeScript + Vite, Tailwind CSS, React Router, TanStack Query, Recharts, React Hook Form + Zod.

Backend : Python 3.11, FastAPI, Pydantic v2, pandas, numpy, statsmodels (lightweight only; NO Prophet/TensorFlow/PyTorch - Vercel size and cold-start limits).

Database : MongoDB Atlas via Motor (async). Create indexes explicitly.

Auth : FastAPI-native JWT (access + refresh), bcrypt/argon2 password hashing, OAuth2PasswordBearer, per-user data isolation.

Deploy : Vercel (frontend static build; FastAPI as Python serverless function under /api via vercel.json). Keep each request < 10 s and stateless; no background threads or local disk persistence.

Config : .env + .env.example, never commit secrets.

---

3. FUNCTIONAL REQUIREMENTS

F1 Auth: register, login, logout, refresh, protected routes. Each user owns one or more stores; all queries are scoped by user_id/store_id.

F2 Sales upload (CSV/XLSX):

- Columns: date, product (name or SKU), quantity_sold; optional: price.
- Column-mapping step (auto-detect headers, let user correct).
- Validate: date parsing (multiple formats, DD/MM vs MM/DD prompt), negative/zero/non-numeric qty, duplicates, empty rows, file size limit (5 MB), minimum history (warn if < 14 days per product).
- Show a preview + row-level error report; allow import of valid rows.
- Provide downloadable sample CSV template.

F3 Product master (inline editable table + CSV import):
name/SKU, unit, current stock on hand, supplier, lead_time_days, shelf_life_days (null = non-perishable), cost_price, selling_price, min_order_qty (MOQ), pack_size (order multiple), optional batch/expiry entries for stock on hand.
Support quick "update stock" so the daily flow takes seconds.

F4 Settings (per store): review/order cycle days (e.g. order every 3 days), target service level (default 95%), expiry safety buffer (default 20% of shelf life), optional budget cap, currency (default INR), week start.

F5 Forecast engine (see section 4) producing, per product: daily forecast, expected demand until next delivery, confidence band, method used, and an explanation string.

F6 Reorder planner (see section 5): ranked list with exact quantities, reason tags, cost of order, expected stock-out date, profit at risk.

F7 Dashboard:
- KPI cards: items to order now, estimated order value, stock-out risks,

cash locked in slow/overstock items, items expiring soon.

- Ranked reorder table (priority badge Critical/High/Medium/Low, qty, supplier, order-by date, "why" tooltip). Filter by supplier, search, sort. Per-supplier grouped view.
- Product detail drawer: sales history chart, forecast with confidence band, stock projection line vs reorder point, shelf-life marker.
- "Do NOT reorder" panel: overstocked / dead / near-expiry items with suggested action (discount, bundle, hold).
- Export order list as CSV and print-friendly PDF; per-supplier WhatsApp /text share of the order.
- Order-list override: owner can edit a quantity; system recomputes totals and keeps the original recommendation visible.

F8 Feedback loop (lightweight): owner marks list as "ordered"; store this; later show forecast-vs-actual accuracy (WAPE/MAPE) per product.

F9 Empty/loading/error states everywhere; responsive mobile-first UI; accessible (labels, contrast, keyboard nav).

---

4. FORECASTING SPEC (must be correct, explainable, and fast)

Pre-processing

- Aggregate to daily per product; fill missing days with 0 ONLY if the product was active (between first sale and last date); treat stock-out days as censored when stock data exists (do not under-forecast).
- Winsorize outliers (e.g. IQR/MAD cap) so one bulk sale does not skew.
- Detect and use day-of-week seasonality when >= 4 weeks of history.

Model selection per product (automatic, with fallback chain)

1. Intermittent demand (>= ~40% zero days): Croston / SBA (Syntetos-Boylan).
2. >= 8 weeks history with weekly pattern: Holt-Winters / ETS (weekly).
3. 14-56 days: Exponential smoothing with day-of-week index.
4. < 14 days or new item: weighted moving average / category average, flagged "Low confidence".

- Choose via rolling-origin backtest (last 2-4 weeks holdout) using WAPE/MAE; keep the winner; store method + error.

Outputs per product

- mean daily forecast d(t) for the horizon, demand std dev from backtest residuals, and an explainable sentence ("Sells ~12/day, more on weekends").

Horizon

H = lead_time_days + review_cycle_days (time until the NEXT delivery after the following order opportunity). Demand_H = sum of d(t) over H.

---

5. REORDER LOGIC (the heart of the product)

Inputs: forecast, stock on hand (S0), open/in-transit orders, lead time L, review cycle R, shelf life SL, MOQ, pack size, costs, prices, service level.

Steps

1. Safety stock SS = z(service_level) * sigma_daily * sqrt(L + R)
2. Reorder point ROP = demand over L + SS
3. Order-up-to level U = Demand_(L+R) + SS

4. Shelf-life cap (critical): the store must not hold more than it can sell before expiry. Usable_days = SL * (1 - expiry_buffer) - L_remaining_age. Cap U at forecast demand over Usable_days. If existing stock will expire before it sells (projected sales < stock before expiry), flag "expiry risk" and subtract the unsellable portion from effective stock.
5. Effective stock = on_hand_sellable + in_transit - projected expiry loss.
6. Raw qty = max(0, U - effective stock).
7. Apply constraints: round UP to pack_size multiples, enforce MOQ (if raw qty > 0 and < MOQ, order MOQ only if it does not breach the shelf-life cap, else flag "MOQ too large for shelf life"), never negative.
8. Optional budget cap: if total > budget, allocate greedily by profit-at-risk per rupee spent; show what was deferred.
9. Return integer units, order value, expected stock-out date, order-by date (stock-out date minus lead time).

Priority ranking score (0-100, transparent and configurable weights)

urgency = based on days_of_cover vs lead_time (cover < L => max)

profit_risk = unit_margin * expected_units_lost_if_not_ordered

velocity = share of total revenue/units (fast movers weigh more)

Score = 0.45*urgency + 0.35*profit_risk_norm + 0.20*velocity_norm

Penalty: reduce score for overstock/near-expiry items.

Labels: Critical (will stock out before delivery), High, Medium, Low.

Sort: Critical first, then by score desc. Show the factor breakdown.

Edge cases to handle explicitly

- Zero sales history, brand-new product, stock = 0, negative stock input, lead time = 0, very long lead time > shelf life (flag impossible), seasonal spike/festival (allow manual demand multiplier per product),

product with no shelf life, price < cost, duplicate product names,
holiday/closed days, unit mismatch, huge quantity outliers.

---

6. DATA MODEL (MongoDB collections + indexes)

---

users(email unique, password_hash, created_at)

stores(user_id, name, settings{service_level, review_days, buffer, budget,
currency})

products(store_id, sku, name, unit, stock_on_hand, supplier_id, lead_time_days,
shelf_life_days, cost, price, moq, pack_size, demand_multiplier,
batches[{qty, expiry}], updated_at) idx: (store_id, sku) unique

suppliers(store_id, name, contact, default_lead_time)

sales(store_id, product_id, date, qty, price) idx: (store_id, product_id, date)

upload_jobs(store_id, filename, status, rows_ok, rows_failed, errors[])

forecasts(store_id, product_id, run_id, method, horizon, mean, sigma, wape,
created_at)

reorder_plans(store_id, run_id, items[{product_id, qty, score, label, reason,
stockout_date, order_by, value}], total_value, created_at, status)

---

7. API (FastAPI, versioned /api/v1, OpenAPI documented)

POST /auth/register, /auth/login, /auth/refresh GET /auth/me

CRUD /stores, /products, /suppliers

POST /sales/upload (multipart) -> job id; GET /sales/jobs/{id}

GET /sales/template

PUT /settings

POST /plan/generate -> runs forecast + reorder, returns plan

GET /plan/latest, GET /plan/{id}, PATCH /plan/{id}/items/{pid} (override qty)

POST /plan/{id}/mark-ordered

GET /products/{id}/forecast (history + forecast + bands + projection)

GET /plan/{id}/export?format=csv|pdf

GET /health

Use consistent error schema {code, message, details}, pagination, request validation, CORS locked to the frontend origin, and rate limiting on auth.

---

8. INDUSTRY EXPECTATIONS (non-negotiable)

Security : hashed passwords, JWT expiry + refresh, input validation, file type/size checks, no secrets in repo, tenant isolation tests.

Quality : strict TypeScript (no any), ESLint + Prettier, Ruff + Black + mypy on backend, typed Pydantic schemas, clear module layout (routers / services / forecasting / repositories / schemas).

Testing : pytest unit tests for forecasting + reorder math (known-answer cases, edge cases above), API tests, Vitest for key components. Target >= 80% coverage on the forecasting/reorder modules.

Reliability: idempotent uploads, atomic plan generation, graceful failures with actionable messages, structured logging, /health endpoint.

Performance: plan for 500 products x 12 months of daily data in < 8 s; use vectorized pandas, bulk Mongo writes, cached results, pagination.

UX : mobile-first, plain language, explain every number ("why this quantity"), undo for edits, onboarding wizard (upload -> map -> set stock -> see plan), sample data button for instant demo.

Docs : README (setup, env vars, architecture diagram, formulas used, assumptions, limitations), API docs, demo script.

DevEx : seed script with a realistic demo dataset (60 products, 6 months, weekly seasonality, intermittent items, perishables, a few stock-outs and overstocks) and sample CSV templates.

Deploy : working Vercel config, env setup guide, CI (lint + tests) via GitHub Actions.

---

9. BUILD PLAN (execute in order)

Phase 0 Scaffold monorepo (/frontend, /backend), configs, env, lint, CI, Vercel config, Mongo connection + indexes.

Phase 1 Auth + store + settings (backend + login/register UI).

Phase 2 Product master + supplier CRUD + CSV import + editable table.

Phase 3 Sales upload pipeline: mapping, validation, preview, error report.

Phase 4 Forecast engine + backtesting + model selection + unit tests.

Phase 5 Reorder engine: safety stock, shelf-life cap, MOQ/pack, ranking, budget allocation + exhaustive unit tests.

Phase 6 Dashboard UI: KPIs, ranked list, supplier view, product drawer with charts, "do not reorder" panel, overrides, export CSV/PDF.

Phase 7 Feedback loop (ordered status, accuracy metrics), onboarding wizard, seed/demo data, empty/error states, accessibility pass.

Phase 8 Hardening: security review, performance test, logging, docs, deploy to Vercel, end-to-end smoke test, demo script (3 minutes).

---

10. ACCEPTANCE CRITERIA (definition of done)

[ ] Upload sample sales + enter stock/lead time/shelf life -> ranked list in under 10 seconds with integer quantities.

[ ] Fast-mover about to stock out within lead time ranks Critical and #1.

[ ] Perishable item never recommended beyond what sells before expiry.

[ ] Slow/dead item gets quantity 0 and appears in "Do not reorder" with cash locked amount.

[ ] Quantities respect MOQ and pack size; totals match line items.

[ ] Every row has a plain-language explanation and factor breakdown.

[ ] Bad files produce clear row-level errors, never a crash.

[ ] One user can never see another user's data (tested).

[ ] Forecast backtest accuracy shown; baseline (naive moving average) is beaten on demo data and reported honestly.

[ ] Deployed Vercel URL works; README lets a stranger run it in 10 minutes.

---

11. OUTPUT FORMAT FOR EACH PHASE

- Short plan of the phase
- Complete code (no placeholders / TODO stubs)
- Commands to run and test
- Acceptance checks result
- Known limitations + next phase

Begin with Phase 0 now. State your assumptions in 5 lines max, then build.

==========

END OF PROMPT

==========

[12:31 pm, 23/10/2026] +91 80156 16233:
==========

STOCKSENSE - LANDING PAGE, AUTH AND APPLICATION | QUANTIX 2K26 | PS11

BUILD PROMPT

==========

ROLE

You are a senior full-stack engineer. Build a production-quality web app named "StockSense" in the order given below. Every screen must work end to end with real data, not placeholders. Work phase by phase. After each phase, list the files created, the commands to run, and the checks you verified.

STACK (fixed)

Frontend : React + TypeScript + Vite, Tailwind CSS, React Router, TanStack Query, React Hook Form + Zod, Recharts.

Backend : Python 3.11, FastAPI, Pydantic v2, Motor (async MongoDB driver),

...

[12:31 pm, 23/10/2026] +91 80156 16233: PS11 – StockSense: Small-Retail Reorder Planner

The problem: Small store owners order from memory. Fast-moving items run out while slow or short-dated items sit on shelves and lock up cash.

What to build: A tool that forecasts demand per product and produces a ranked reorder list with exact quantities, respecting supplier lead time and shelf life.

Explanation: The owner uploads past sales and enters stock on hand, supplier lead time and shelf life. The tool predicts how much of each product will sell until the next

delivery. It then produces an order list ranked by importance, with the exact number of units to buy for each item, so the most urgent and most profitable items come first.